UNIVERSIDAD RAFAEL LANDÍVAR

FACULTAD DE INGENIERÍA

LICENCIATURA EN INGENIERIA EN INFORMATICA Y SISTEMAS

MANEJO E IMPLEMENTACION DE ARCHIVOS

PROYECTO 2

"MANUAL TECNICO"

INTEGRANTE:	**CARNET:**
Klitheryn Karos Fernanda Zapeta Felipe	1516925

QUETZALTENANGO, SEPTIEMBRE DE 2026

CAMPUS SAN ALBERTO HURTADO

1. Estructura del Registro

Para que el acceso aleatorio funcione, todos los registros dentro del archivo contactos.dat deben ocupar exactamente la misma cantidad de memoria física (bytes). Se utilizó el módulo struct de Python para empaquetar las variables en formato binario.

Largo de cada campo y formato:

- **ID:** Entero de 4 bytes (i).
- **Nombre:** Cadena de 50 bytes (50s).
- **Teléfono:** Cadena de 15 bytes (15s).
- **Correo:** Cadena de 40 bytes (40s).
- **Ciudad:** Cadena de 30 bytes (30s).

Tamaño total del registro: Al sumar todos los campos, cada registro ocupa exactamente 139 bytes. Para garantizar que los textos cortos no corrompan el tamaño, se utiliza el método .ljust() rellenando el espacio sobrante con bytes nulos (\x00).

2. Implementación del Acceso Aleatorio (seek y tell)

El acceso aleatorio evita el desperdicio de recursos al impedir que el sistema deba leer todo el archivo para llegar a un dato específico.

Uso de tell()

El método tell() devuelve la posición actual del puntero en bytes. Al agregar un registro, el puntero se mueve al final del archivo (ab). Con tell() obtenemos esa posición en bytes y la dividimos por 139 (el tamaño del registro) para calcular el número de registro lógico que se asignará al contacto.

Fragmento de código:

```
def agregar(self, contacto: Contacto) -> int:
  with open(self.ruta, "ab") as archivo:
    posicion_byte = archivo.tell() # Indica en qué byte exacto estamos
    archivo.write(contacto.empaquetar())
    num_registro = posicion_byte // TAMANIO # Calculamos el id lógico (0, 1, 2...)
    return int(num_registro)
```

Uso de seek()

El método seek(offset) permite mover el puntero de lectura/escritura directamente a un byte en específico. Se multiplica el número de registro por 139 bytes para saber dónde inicia el contacto deseado. A partir de esa posición, se leen o sobrescriben únicamente los 139 bytes, logrando una **modificación en el lugar** sin reescribir todo el .dat.

Fragmento de código:

```
def modificar_registro(self, num_registro: int, contacto: Contacto):
  with open(self.ruta, "r+b") as archivo:
    posicion_byte = num_registro * TAMANIO
    archivo.seek(posicion_byte) # Salto directo en memoria
    archivo.write(contacto.empaquetar()) # Sobrescritura en el lugar exacto
```

3. El Índice Primario y la Eficiencia en Búsquedas

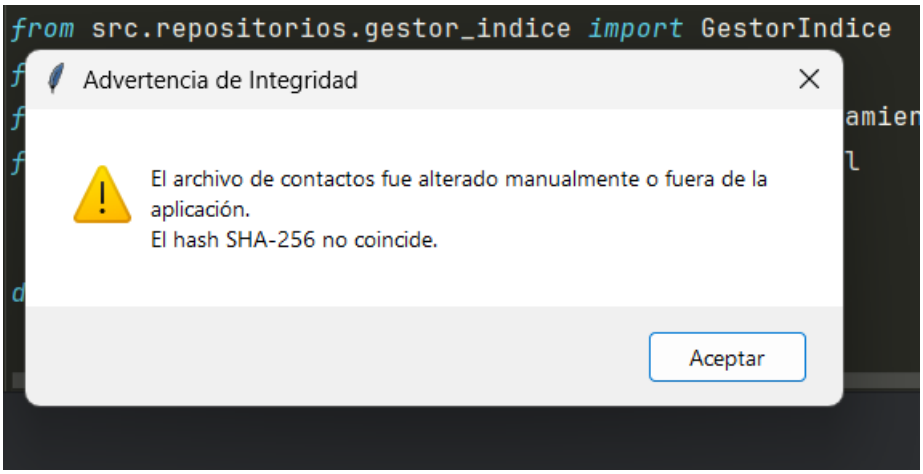
El sistema mantiene un archivo indice.json que asocia el ID ingresado por el usuario con el número de registro físico calculado con tell().

- **Búsqueda Secuencial:** Si no existiera el índice, buscar el contacto con ID 1000 obligaría al disco duro a iterar y desempaquetar registro por registro desde el inicio hasta encontrarlo. Su complejidad de tiempo es O(N).
- **Búsqueda con Índice:** Al arrancar el programa, el archivo JSON se carga en la memoria RAM como un Diccionario (Tabla Hash). Al buscar un ID, el diccionario devuelve el número de registro físico casi de forma instantánea. Su complejidad de tiempo es O(1). Una vez tenemos el número, se hace un solo salto con seek(). Es infinitamente más rápido y no desgasta el disco.

4. Evidencia de Verificación de Integridad

Al iniciar el programa, la clase HerramientasArchivos lee los bytes de contactos.dat, calcula su hash SHA-256 y lo compara contra el hash guardado en contactos.sha256.

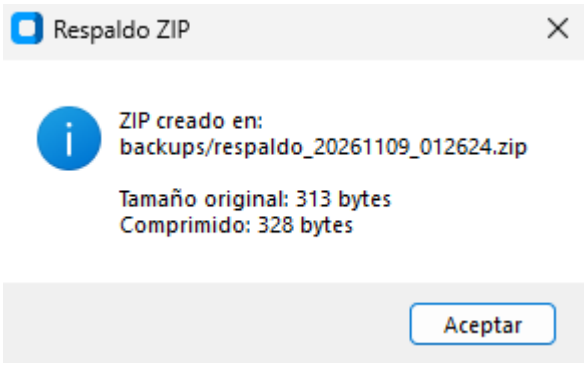
(A continuación se muestra la detección del sistema tras alterar el archivo manualmente en el bloc de notas):



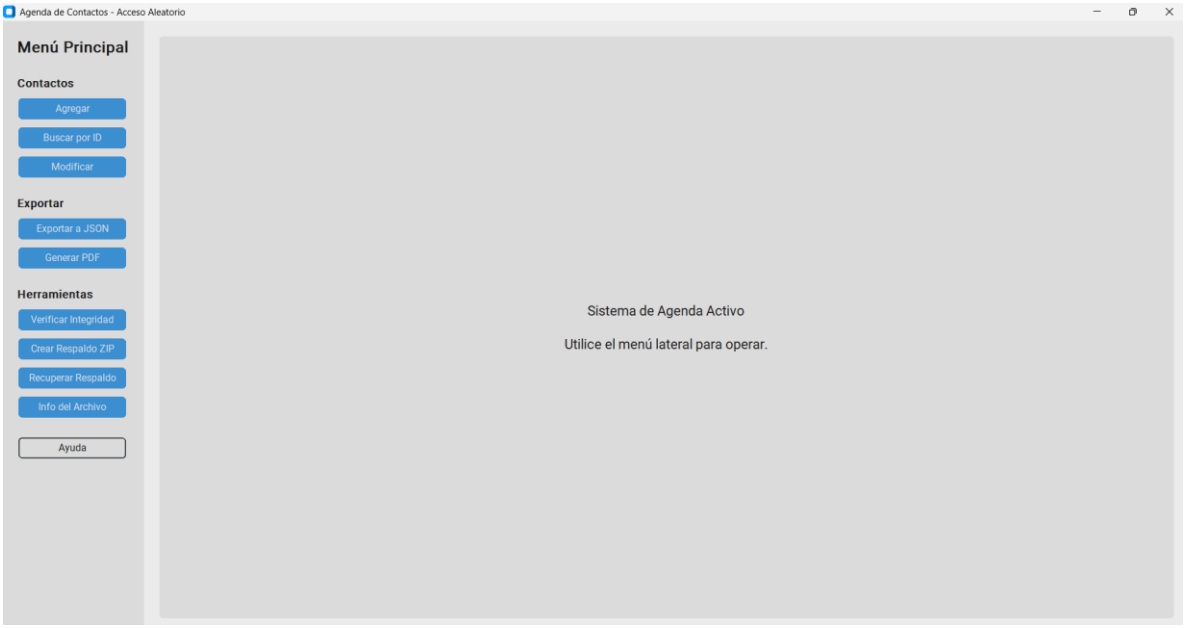
5. Comparación de Tamaño: Original vs Respaldo ZIP

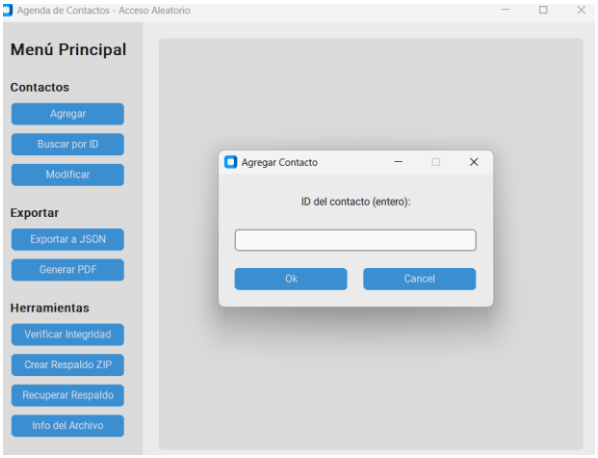
El sistema cuenta con una opción de respaldo que comprime el archivo binario y el índice utilizando el módulo zipfile con el algoritmo ZIP_DEFLATED.

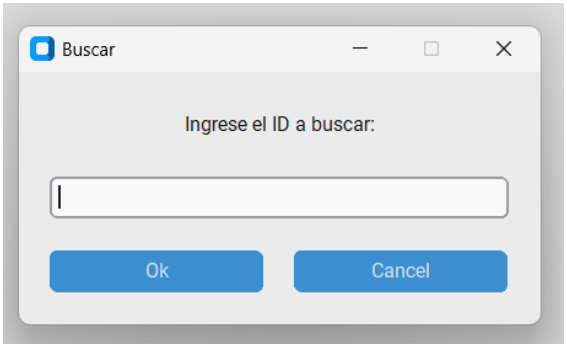
(Se demuestra la reducción de tamaño tras crear el respaldo):

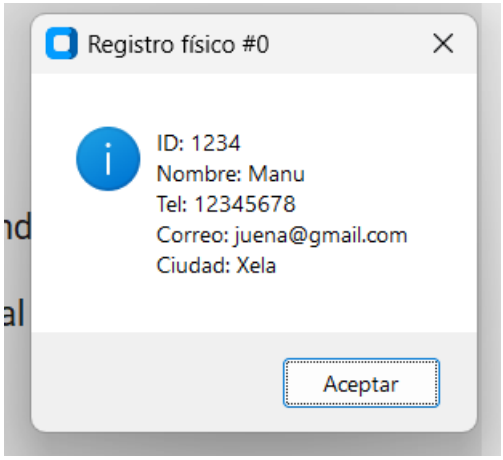


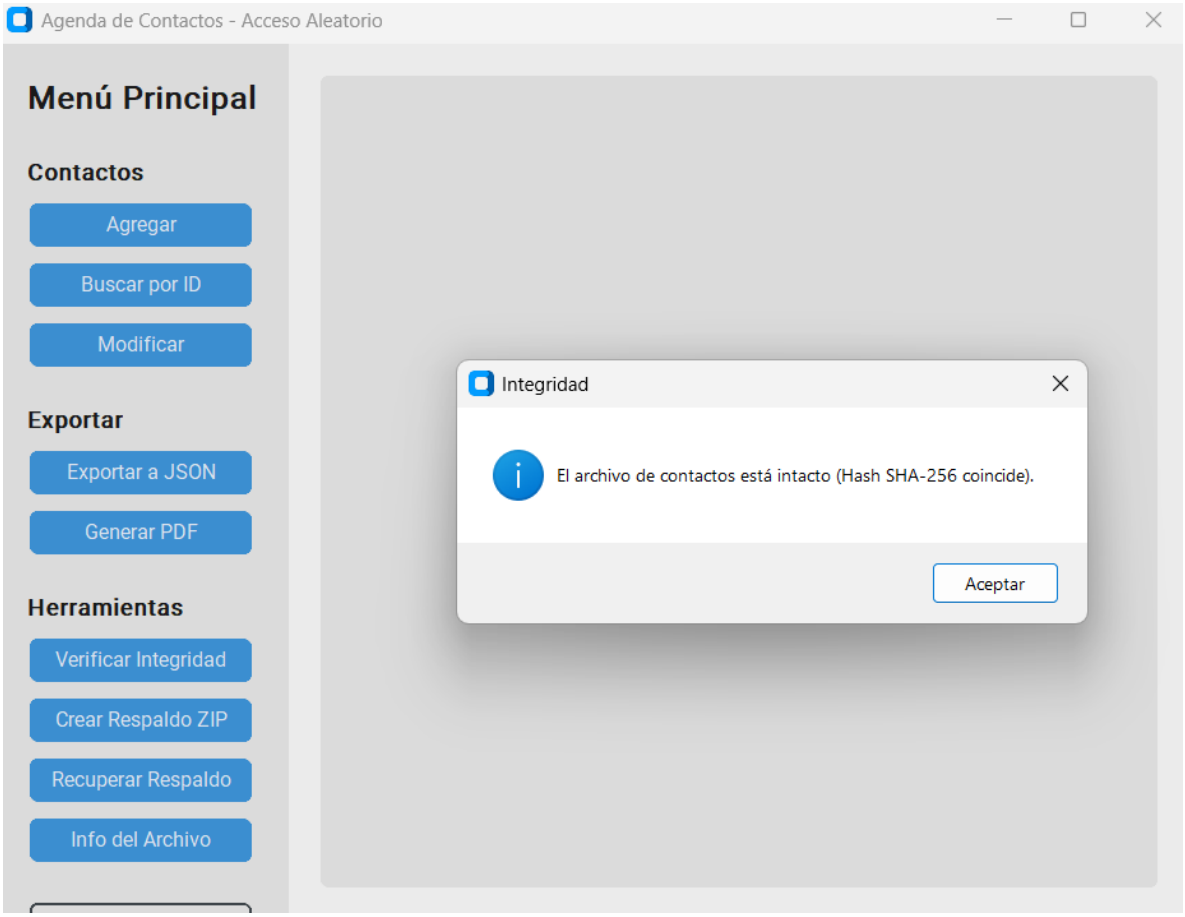
6. Capturas de Pantalla del Funcionamiento

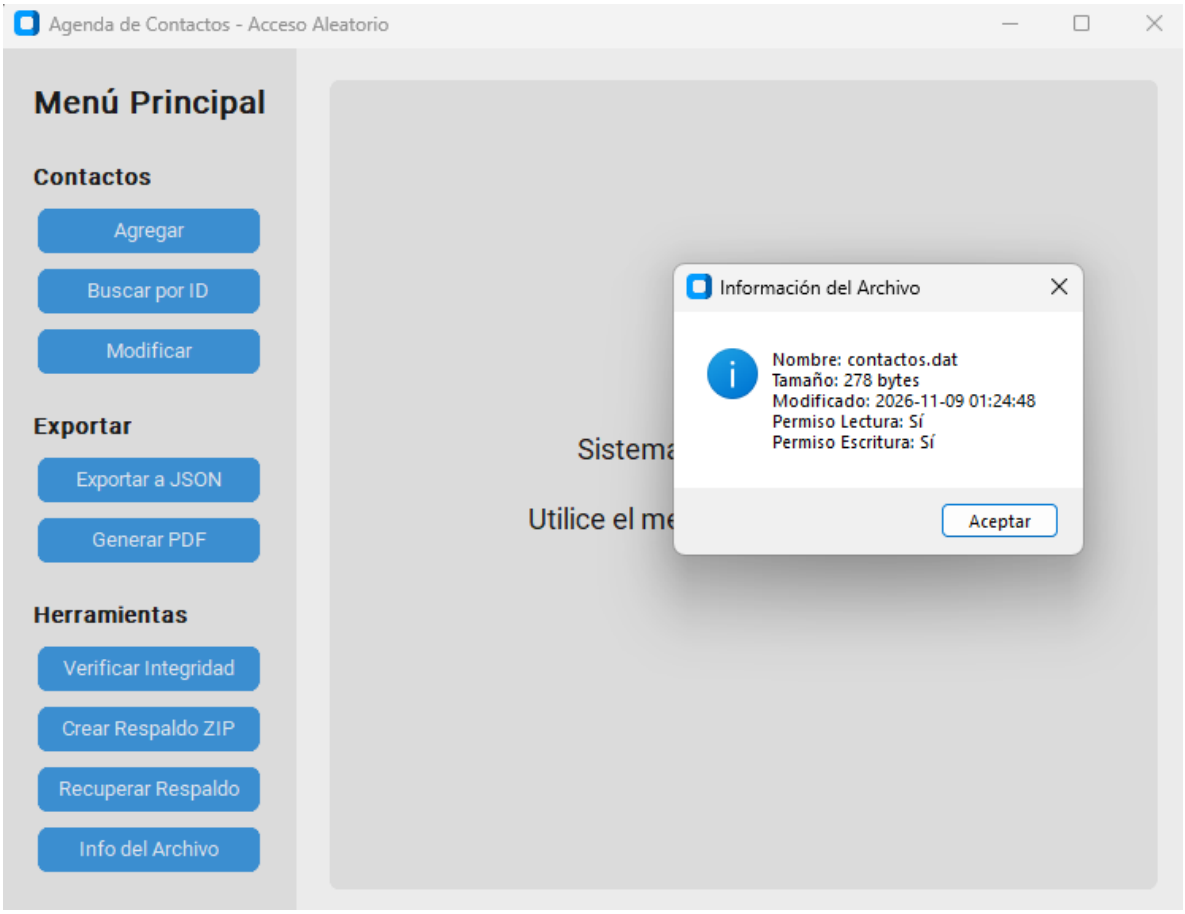

Agenda de Contactos - Acceso Aleatorio
Menú Principal
Contactos
Agregar
Buscar por ID
Modificar
Exportar
Exportar a JSON
Generar PDF
Herramientas
Verificar Integridad
Crear Respaldo ZIP
Recuperar Respaldo
Info del Archivo
Ayuda
Sistema de Agenda Activo
Utilice el menú lateral para operar.


Agenda de Contactos - Acceso Aleatorio
Menú Principal
Contactos
Agregar
Buscar por ID
Modificar
Exportar
Exportar a JSON
Generar PDF
Herramientas
Verificar Integridad
Crear Respaldo ZIP
Recuperar Respaldo
Info del Archivo
Agregar Contacto
ID del contacto (entero):
Ok
Cancel


Buscar
Ingrese el ID a buscar:
Ok
Cancel


Registro físico #0
ID: 1234
Nombre: Manu
Tel: 12345678
Correo: juena@gmail.com
Ciudad: Xela
Aceptar


Agenda de Contactos - Acceso Aleatorio
Menú Principal
Contactos
Agregar
Buscar por ID
Modificar
Exportar
Exportar a JSON
Generar PDF
Herramientas
Verificar Integridad
Crear Respaldo ZIP
Recuperar Respaldo
Info del Archivo
Integridad
El archivo de contactos está intacto (Hash SHA-256 coincide).
Aceptar


Agenda de Contactos - Acceso Aleatorio
Menú Principal
Contactos
Agregar
Buscar por ID
Modificar
Exportar
Exportar a JSON
Generar PDF
Herramientas
Verificar Integridad
Crear Respaldo ZIP
Recuperar Respaldo
Info del Archivo
Sistema
Utilice el me
Información del Archivo
Nombre: contactos.dat
Tamaño: 278 bytes
Modificado: 2026-11-09 01:24:48
Permiso Lectura: Sí
Permiso Escritura: Sí
Aceptar